

Embedded Linux & System Software Interview Handbook

Chapter 1: Computer Architecture Fundamentals

Goal: Understand the hardware building blocks before learning Boot Process, BSP, Board Bring-up and Embedded Linux.

Computer Architecture

A typical embedded system consists of CPU, RAM, Flash, Memory Controller, Interrupt Controller and peripherals like UART, I2C, SPI, GPIO and USB.

CPU

The CPU follows the Fetch → Decode → Execute cycle. Modern processors improve performance using pipelining, branch prediction and out-of-order execution.

Registers

Registers are the fastest storage in the CPU. ARM examples: R0-R15, SP, LR, PC. x86 examples: RAX, RBX, RCX, RDX, RSP, RIP.

Memory Hierarchy

Registers → L1 Cache → L2 Cache → L3 Cache → RAM → SSD/eMMC/NAND. Access time increases at each lower level.

Cache

Caches reduce memory latency using temporal and spatial locality. L1 is smallest and fastest, while L3 is larger and shared among cores.

RAM, ROM and Flash

RAM stores executing code and data. Flash stores persistent software such as U-Boot, Linux kernel and RootFS. BootROM starts execution immediately after power-on.

Stack vs Heap

The stack is automatically managed and stores local variables and function frames. The heap is dynamically allocated and suitable for runtime-sized objects.

Interrupts

Interrupts allow peripherals to notify the CPU. The CPU saves context, executes the Interrupt Service Routine (ISR), and resumes the interrupted task.

Samsung Smart TV Example

UART was used during board bring-up to capture early boot logs and diagnose DDR initialization failures, bootloader issues, kernel panics and driver initialization problems.

Interview Questions

Explain the Fetch-Decode-Execute cycle.

Why are registers faster than RAM?

Explain memory hierarchy.

Difference between RAM, ROM and Flash.

What is cache?

Stack vs Heap.

What is an interrupt and ISR?

Why are interrupts preferred over polling?

Key Takeaways

Master CPU fundamentals, memory hierarchy, cache, RAM, ROM, Flash, stack, heap and interrupts. These concepts are essential for understanding Boot Process, BSP and Board Bring-up.